Joyce Maketa

Lab 01

Isolation réseau : Création de trois zones étanches (WAN, DMZ, LAN).

Filtrage Stateful : Le pare-feu laisse passer le trafic, il se souvient également des connexions (ESTABLISHED).

Routage statique : Chemin de retour est aussi important que l'aller.

Création des réseaux Docker

```
joyce@Docker: ~/dmz-docke
joyce@Docker:~$ mkdir dmz-docker-lab
cd dmz-docker-lab
joyce@Docker:~/dmz-docker-lab$ docker network create --subnet 172.30.0.0/24 wan_net
docker network create --subnet 172.20.0.0/24 dmz_net
docker network create --subnet 172.10.0.0/24 lan_net
50a4d71bd6df9e29af9fea28319df528b7087e65700e2d3212ca0b067cde351c
3a57c5682c5ebaa4b54f82fc3a0e7762f42b3256177a5f8b94248c736b4d716f
d78db81bc71fb6198b43f54891a163e2cc7d8a756c62d7eaacca79a7e89378ec
joyce@Docker:~/dmz-docker-lab$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
6e794c66a719   bridge    bridge    local
3a57c5682c5e   dmz_net   bridge    local
06aab3438815   host      host      local
d78db81bc71f   lan_net   bridge    local
d4c2d7effd36   none      null      local
50a4d71bd6df   wan_net   bridge    local
joyce@Docker:~/dmz-docker-lab$ docker network inspect dmz_net
[
    {
        "Name": "dmz_net",
        "Id": "3a57c5682c5ebaa4b54f82fc3a0e7762f42b3256177a5f8b94248c736b4d716f",
        "Created": "2026-02-09T20:19:18.501690394Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
```

Création du conteneur Routeur

```
    }
]
joyce@Docker:~/dmz-docker-lab$ docker run -dit --name routeur --cap-add=NET_ADMIN alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
d1ea5d684c3b109f7f32cb464b02131d1bba1e3fdafa478d9f9dbf749725a809
joyce@Docker:~/dmz-docker-lab$ docker network connect wan_net routeur
docker network connect dmz_net routeur
docker network connect lan_net routeur
joyce@Docker:~/dmz-docker-lab$ docker exec -it routeur apk add iptables
```

Installation des outils réseau :

3. Mise en place du pare-feu / Routeur,

```
(4/4) Installing iptables (1.8.11-r1)
Executing busybox-1.37.0-r30.trigger
OK: 10.1 MiB in 20 packages
joyce@Docker:~/dmz-docker-lab$ docker exec -it routeur sh
/ # echo 1 > /proc/sys/net/ipv4/ip_forward
sh: can't create /proc/sys/net/ipv4/ip_forward: Read-only file system
/ #
```

« erreur can't create /proc/sys/net/ipv4/ip_forward » après avoir essayé d'activer le routage, mais Docker bloque l'accès car c'est une modification sensible du noyau.

```
/ # exit
joyce@Docker:~/dmz-docker-lab$ docker rm -f routeur
routeur
joyce@Docker:~/dmz-docker-lab$ docker run -dit --name routeur --privileged alpine sh
a3a0c9274b8f4b9647260bf10984016af5aadeb9aaa8bf86b9f8d51f3b83a61f
joyce@Docker:~/dmz-docker-lab$ docker network connect wan_net routeur
joyce@Docker:~/dmz-docker-lab$ docker network connect dmz_net routeur
joyce@Docker:~/dmz-docker-lab$ docker network connect lan_net routeur
joyce@Docker:~/dmz-docker-lab$ docker exec -it routeur apk add iptables
docker exec -it routeur sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
(1/4) Installing libmnl (1.0.5-r2)
(2/4) Installing libnftnl (1.3.0-r0)
(3/4) Installing libxtables (1.8.11-r1)
(4/4) Installing iptables (1.8.11-r1)
Executing busybox-1.37.0-r30.trigger
OK: 10.1 MiB in 20 packages
joyce@Docker:~/dmz-docker-lab$
```

4. Mise en place des serveurs (Web et BDD).

```
joyce@Docker:~/dmz-docker-lab$ docker run -dit --name serveur-bdd --network lan_net alpine sh
46f15f238f9d19692a8a3174fdbc09dbfca420d828d2dda5f26326262a85396e
joyce@Docker:~/dmz-docker-lab$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' serveur-web
172.20.0.3
joyce@Docker:~/dmz-docker-lab$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' serveur-bdd
172.10.0.3
joyce@Docker:~/dmz-docker-lab$
```

4. Configuration du pare-feu (iptables) : Vérification des interfaces.

Il faut que eth0, eth1 et eth2 correspondent aux bons réseaux pour que les futures règles iptables ne bloquent pas le mauvais trafic.

```
joyce@Docker:~/dmz-docker-lab$ docker exec -it routeur sh
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0@if15: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether de:a2:46:82:0e:cb brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
       valid_lft forever preferred_lft forever
3: eth1@if16: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 4a:80:e1:8d:2f:19 brd ff:ff:ff:ff:ff:ff
    inet 172.30.0.2/24 brd 172.30.0.255 scope global eth1
       valid_lft forever preferred_lft forever
4: eth2@if17: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 56:32:c8:2f:91:c1 brd ff:ff:ff:ff:ff:ff
    inet 172.20.0.2/24 brd 172.20.0.255 scope global eth2
       valid_lft forever preferred_lft forever
5: eth3@if18: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether c6:73:d6:ea:e5:42 brd ff:ff:ff:ff:ff:ff
    inet 172.10.0.2/24 brd 172.10.0.255 scope global eth3
       valid_lft forever preferred_lft forever
/ #
```

```
/ # iptables -F
 INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT/ # iptables -X
/ #
/ # # On bloque tout par défaut (Politique DROP)
/ # iptables -P INPUT DROP
/ # iptables -P FORWARD DROP
/ # iptables -P OUTPUT ACCEPT
/ #
```

Règles de transfert (FORWARD) pour autoriser le trafic

```
/ # iptables -P INPUT DROP
/ # iptables -P FORWARD DROP
/ # iptables -P OUTPUT ACCEPT
/ # iptables -A FORWARD -i eth1 -o eth2 -p tcp --dport 80 -j ACCEPT
/ # iptables -A FORWARD -i eth2 -o eth1 -m state --state ESTABLISHED,RELATED -j ACCEPT
/ # iptables -A FORWARD -i eth2 -o eth3 -p tcp --dport 3306 -j ACCEPT
/ # iptables -A FORWARD -i eth3 -o eth2 -m state --state ESTABLISHED,RELATED -j ACCEPT
/ # iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     tcp  --  eth1   eth2    0.0.0.0/0            0.0.0.0/0            tcp dpt:80
    0     0 ACCEPT     all  --  eth2   eth1    0.0.0.0/0            0.0.0.0/0            state RELATED,ESTABLISHED
    0     0 ACCEPT     tcp  --  eth2   eth3    0.0.0.0/0            0.0.0.0/0            tcp dpt:3306
    0     0 ACCEPT     all  --  eth3   eth2    0.0.0.0/0            0.0.0.0/0            state RELATED,ESTABLISHED

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
/ #
```

5. Déploiement du serveur Web (DMZ) :

```
joyce@Docker:~/dmz-docker-lab$ docker run -dit --name web_dmz --network dmz_net nginx
d1d82af6e9f0b5708d23a471b5c3449c2822a824cb8a988c81973af047dc1e6c
joyce@Docker:~/dmz-docker-lab$ docker inspect web_dmz | grep "IPAddress"
            "SecondaryIPAddresses": null,
            "IPAddress": "",
                    "IPAddress": "172.20.0.4",
joyce@Docker:~/dmz-docker-lab$ docker run -dit --name bdd_lan --network lan_net alpine sh
a6ab5f7b7d28ec570484cb234b4092cde70740c144e3770fa98705791d69b121
joyce@Docker:~/dmz-docker-lab$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS     NAMES
a6ab5f7b7d28   alpine    "sh"                     8 seconds ago        Up 6 seconds                  bdd_lan
d1d82af6e9f0   nginx     "/docker-entrypoint.…"   About a minute ago   Up About a minute   80/tcp    web_dmz
46f15f238f9d   alpine    "sh"                     10 minutes ago       Up 10 minutes                 serveur-bdd
e3903b8c5db6   nginx     "/docker-entrypoint.…"   11 minutes ago       Up 11 minutes       80/tcp    serveur-web
a3a0c9274b8f   alpine    "sh"                     16 minutes ago       Up 16 minutes                 routeur
joyce@Docker:~/dmz-docker-lab$
```

```
joyce@Docker:~/dmz-docker-lab$ docker exec web_dmz curl localhost
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 <!DOCTYPE html>0     0     0      0      0 --:--:-- --:--:-- --:--:--     0
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
100   615  100   615    0     0   4960      0 --:--:-- --:--:-- --:--:--  5000
joyce@Docker:~/dmz-docker-lab$
```

6. Déploiement des clients :

```
joyce@Docker:~/dmz-docker-lab$ docker exec -it lan_client curl 1
72.20.0.4
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully in
stalled and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
joyce@Docker:~/dmz-docker-lab$
```

Bug rencontré, "Operation not permitted" & "File not found"

Ce problème était un conflit de droits et de configuration au moment de finaliser le routage retour.

Lorsque j'ai a essayé d'ajouter la route avec ip route add, Docker a répondu Operation not permitted car le serveur n'avait pas l'autorisation de modifier sa propre table de routage. J'ai ensuite recréé le serveur pour lui donner les droits, l'outil ip n'était plus là (File not found) car dans Docker, si on supprime un conteneur, toutes les installations faites dedans (comme apt install iproute2) disparaissent avec lui.

Résolution :

- Recréation du serveur web_dmz avec l'option --cap-add NET_ADMIN.
- Réinstallation immédiate du paquet iproute2.
- Application de la route statique vers le LAN via l'IP du firewall.

Le serveur Web ne possédant pas de route pour le sous-réseau LAN (172.10.0.0/24), les paquets de réponse HTTP étaient abandonnés. La résolution a nécessité l'élévation des privilèges du conteneur (NET_ADMIN) pour permettre l'ajout d'une route statique pointant vers l'interface DMZ du firewall.

Commandes utilisées

```
joyce@Docker:~/dmz-docker-lab$ history
    1  docker run hello-world
    2  mkdir dmz-docker-lab
    3  cd dmz-docker-lab
    4  docker network create --subnet 172.30.0.0/24 wan_net
    5  docker network create --subnet 172.20.0.0/24 dmz_net
    6  docker network create --subnet 172.10.0.0/24 lan_net
    7  docker network ls
    8  docker network inspect dmz_net
    9  docker run -dit --name routeur --cap-add=NET_ADMIN alpine sh
   10  docker network connect wan_net routeur
   11  docker network connect dmz_net routeur
   12  docker network connect lan_net routeur
   13  docker exec -it routeur apk add iptables
   14  docker exec -it routeur sh
   15  docker rm -f routeur
   16  docker run -dit --name routeur --privileged alpine sh
   17  docker network connect wan_net routeur
   18  docker network connect dmz_net routeur
   19  docker network connect lan_net routeur
   20  docker exec -it routeur apk add iptables
   21  docker exec -it routeur sh -c "echo 1 > /proc/sys/net/ipv4/ip_forward"
   22  docker run -dit --name serveur-web --network dmz_net nginx
   23  docker run -dit --name serveur-bdd --network lan_net alpine sh
   24  docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' serveur-web
   25  docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' serveur-bdd
   26  docker exec -it routeur sh
   27  docker run -dit --name web_dmz --network dmz_net nginx
   28  docker inspect web_dmz | grep "IPAddress"
   29  docker run -dit --name bdd_lan --network lan_net alpine sh
   30  docker ps
   31  docker exec web_dmz curl localhost
   32  docker run -dit --name client_externe --network wan_net alpine sh
   33  docker run -dit --name client_interne --network lan_net alpine sh
   34  docker ps --format "table {{.Names}}\t{{.Networks}}\t{{.Status}}"
   35  docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
   36  docker exec -it lan_client apk update
```

```
   27  docker run -dit --name web_dmz --network dmz_net nginx
   28  docker inspect web_dmz | grep "IPAddress"
   29  docker run -dit --name bdd_lan --network lan_net alpine sh
   30  docker ps
   31  docker exec web_dmz curl localhost
   32  docker run -dit --name client_externe --network wan_net alpine sh
   33  docker run -dit --name client_interne --network lan_net alpine sh
   34  docker ps --format "table {{.Names}}\t{{.Networks}}\t{{.Status}}"
   35  docker run -dit --name lan_client --cap-add NET_ADMIN --network lan_net alpine
   36  docker exec -it lan_client apk update
   37  docker exec -it lan_client apk add iproute2 curl
   38  docker exec -it lan_client sh
   39  docker exec -it lan_client apk update
   40  docker exec -it lan_client apk add iproute2 curl
   41  docker exec -it lan_client curl --version
   42  docker inspect firewall grep 172.10
   43  docker inspect routeur | grep 172.10
   44  docker exec -it lan_client sh
   45  docker exec -it routeur sh
   46  docker exec -it lan_client curl 172.20.0.4
   47  docker exec -it routeur sh
   48  docker exec -it lan_client curl 172.20.0.4
   49  docker exec -it web_dmz apt update && docker exec -it web_dmz apt install iproute2 -y
   50  docker exec -it web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
   51  docker exec -it lan_client curl 172.20.0.4
   52  docker exec -it web_dmz apt update && docker exec -it web_dmz apt install iproute2 -y
   53  docker exec -it web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
   54  docker rm -f web_dmz
   55  docker run -dit --name web_dmz --cap-add NET_ADMIN --network dmz_net nginx
   56  docker exec -it web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
   57  docker exec -it web_dmz apt update && docker exec -it web_dmz apt install iproute2 -y
   58  docker exec -it web_dmz ip route add 172.10.0.0/24 via 172.20.0.2
   59  docker exec -it lan_client curl 172.20.0.4
   60  history
joyce@Docker:~/dmz-docker-lab$
```